

CHAPTER 20

Multiprocessing and Vectorization

20.1 MOTIVATION

Multiprocessing is essential to ML. ML algorithms are computationally intensive, and they will require an efficient use of all your CPUs, servers, and clusters. For this reason, most of the functions presented throughout this book were designed for asynchronous multiprocessing. For example, we have made frequent use of a mysterious function called `mpPandasObj`, without ever defining it. In this chapter we will explain what this function does. Furthermore, we will study in detail how to develop multiprocessing engines. The structure of the programs presented in this chapter is agnostic to the hardware architecture used to execute them, whether we employ the cores of a single server or cores distributed across multiple interconnected servers (e.g., in a high-performance computing cluster or a cloud).

20.2 VECTORIZATION EXAMPLE

Vectorization, also known as array programming, is the simplest example of parallelization, whereby an operation is applied at once to the entire set of values. As a minimal example, suppose that you need to do a brute search through a 3-dimensional space, with 2 nodes per dimension. The un-vectorized implementation of that Cartesian product will look something like Snippet 20.1. How would this code look if you had to search through 100 dimensions, or if the number of dimensions was defined by the user during runtime?

SNIPPET 20.1 UN-VECTORIZED CARTESIAN PRODUCT

```
# Cartesian product of dictionary of lists
dict0={'a':['1','2'],'b':['+','*'],'c':['!','@']}
for a in dict0['a']:
    for b in dict0['b']:
        for c in dict0['c']:
            print {'a':a,'b':b,'c':c}
```

A vectorized solution would replace all explicit iterators (e.g., `For . . . loops`) with matrix algebra operations or compiled iterators or generators. Snippet 20.2 implements the vectorized version of Snippet 20.1. The vectorized version is preferable for four reasons: (1) slow nested `For . . . loops` are replaced with fast iterators; (2) the code infers the dimensionality of the mesh from the dimensionality of `dict0`; (3) we could run 100 dimensions without having to modify the code, or need 100 `For . . . loops`; and (4) under the hood, Python can run operations in C or C++.

SNIPPET 20.2 VECTORIZED CARTESIAN PRODUCT

```
# Cartesian product of dictionary of lists
from itertools import izip, product
dict0={'a':['1','2'],'b':['+','*'],'c':['!','@']}
jobs=(dict(izip(dict0,i)) for i in product(*dict0.values()))
for i in jobs:print i
```

20.3 SINGLE-THREAD VS. MULTITHREADING VS. MULTIPROCESSING

A modern computer has multiple CPU sockets. Each CPU has many cores (processors), and each core has several threads. Multithreading is the technique by which several applications are run in parallel on two or more threads under the same core. One advantage of multithreading is that, because the applications share the same core, they share the same memory space. That introduces the risk that several applications may write on the same memory space at the same time. To prevent that from happening, the Global Interpreter Lock (GIL) assigns write access to one thread per core at a time. Under the GIL, Python's multithreading is limited to one thread per processor. For this reason, Python achieves parallelism through multiprocessing rather than through actual multithreading. Processors do not share the same memory space, hence multiprocessing does not risk writing to the same memory space; however, that also makes it harder to share objects between processes.

Python functions implemented for running on a single-thread will use only a fraction of a modern computer's, server's, or cluster's power. Let us see an example of how a simple task can be run inefficiently when implemented for

single-thread execution. Snippet 20.3 finds the earliest time 10,000 Gaussian processes of length 1,000 touch a symmetric double barrier of width 50 times the standard deviation.

SNIPPET 20.3 SINGLE-THREAD IMPLEMENTATION OF A ONE-TOUCH DOUBLE BARRIER

```
import numpy as np
#-----
def main0():
    # Path dependency: Sequential implementation
    r=np.random.normal(0,.01,size=(1000,10000))
    t=barrierTouch(r)
    return
#-----
def barrierTouch(r,width=.5):
    # find the index of the earliest barrier touch
    t,p={},np.log((1+r).cumprod(axis=0))
    for j in xrange(r.shape[1]): # go through columns
        for i in xrange(r.shape[0]): # go through rows
            if p[i,j]>=width or p[i,j]<=-width:
                t[j]=i
                break
    return t
#-----
if __name__=='__main__':
    import timeit
    print min(timeit.Timer('main0()',setup='from __main__ import main0').repeat(5,10))
```

Compare this implementation with Snippet 20.4. Now the code splits the previous problem into 24 tasks, one per processor. The tasks are then run asynchronously in parallel, using 24 processors. If you run the same code on a cluster with 5000 CPUs, the elapsed time will be about 1/5000 of the single-thread implementation.

SNIPPET 20.4 MULTIPROCESSING IMPLEMENTATION OF A ONE-TOUCH DOUBLE BARRIER

```

import numpy as np
import multiprocessing as mp
#-----
def main1():
    # Path dependency: Multi-threaded implementation
    r,numThreads=np.random.normal(0,.01,size=(1000,10000)),24
    parts=np.linspace(0,r.shape[0],min(numThreads,r.shape[0])+1)
    parts,jobs=np.ceil(parts).astype(int),[]
    for i in xrange(1,len(parts)):
        jobs.append(r[:,parts[i-1]:parts[i]]) # parallel jobs
    pool,out=mp.Pool(processes=numThreads),[]
    outputs=pool.imap_unordered(barrierTouch,jobs)
    for out_ in outputs:out.append(out_) # asynchronous response
    pool.close();pool.join()
    return
#-----
if __name__=='__main__':
    import timeit
    print min(timeit.Timer('main1()',setup='from __main__ import main1').repeat(5,10))

```

Moreover, you could implement the same code to multiprocessing a vectorized function, as we did with function `applyPtS10nT1` in Chapter 3, where parallel processes execute subroutines that include vectorized pandas objects. In this way, you will achieve two levels of parallelization at once. But why stop there? You could achieve three levels of parallelization at once by running multiprocessed instances of vectorized code in an HPC cluster, where each node in the cluster provides the third level of parallelization. In the next sections, we will explain how multiprocessing works.

20.4 ATOMS AND MOLECULES

When preparing jobs for parallelization, it is useful to distinguish between atoms and molecules. Atoms are indivisible tasks. Rather than carrying out all these tasks sequentially in a single thread, we want to group them into molecules, which can be processed in parallel using multiple processors. Each molecule is a subset of atoms that will be processed sequentially, by a callback function, using a single thread. Parallelization takes place at the molecular level.

20.4.1 Linear Partitions

The simplest way to form molecules is to partition a list of atoms in subsets of equal size, where the number of subsets is the minimum between the number of processors and the number of atoms. For N subsets we need to find the $N + 1$ indices that enclose the partitions. This logic is demonstrated in Snippet 20.5.

SNIPPET 20.5 The `linParts` FUNCTION

```
import numpy as np
#-----
def linParts(numAtoms,numThreads):
    # partition of atoms with a single loop
    parts=np.linspace(0,numAtoms,min(numThreads,numAtoms)+1)
    parts=np.ceil(parts).astype(int)
    return parts
```

It is common to encounter operations that involve two nested loops. For example, computing a SADP series (Chapter 17), evaluating multiple barrier touches (Chapter 3), or computing a covariance matrix on misaligned series. In these situations, a linear partition of the atomic tasks would be inefficient, because some processors would have to solve a much larger number of operations than others, and the calculation time will depend on the heaviest molecule. A partial solution is to partition the atomic tasks in a number of jobs that is a multiple of the number of processors, then front-load the jobs queue with the heavy molecules. In this way, the light molecules will be assigned to processors that have completed the heavy molecules first, keeping all CPUs busy until the job queue is depleted. In the next section, we will discuss a more complete solution. [Figure 20.1](#) plots a linear partition of 20 atomic tasks of equal complexity into 6 molecules.

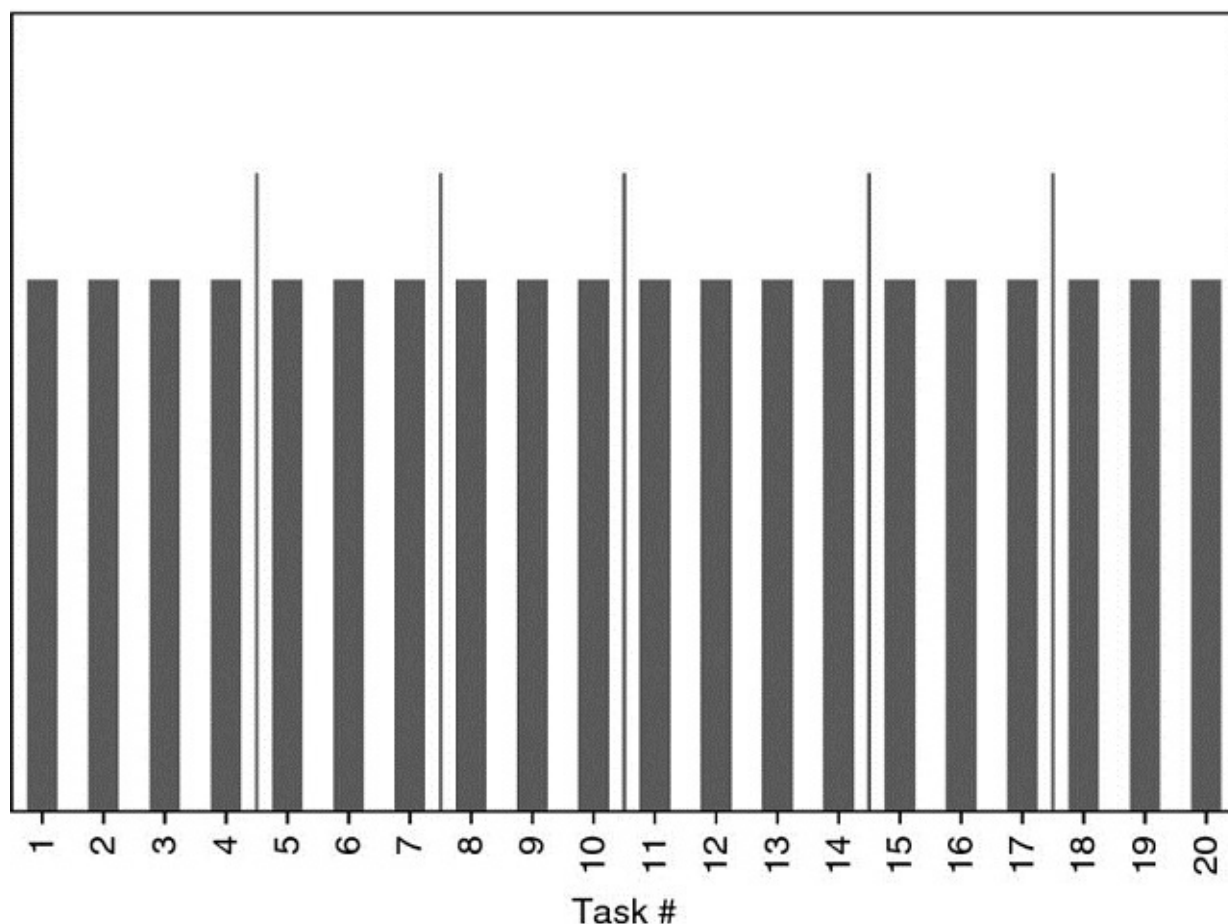


FIGURE 20.1 A linear partition of 20 atomic tasks into 6 molecules

20.4.2 Two-Nested Loops Partitions

Consider two nested loops, where the outer loop iterates $i = 1, \dots, N$ and the inner loop iterates $j = 1, \dots, i$. We can order these atomic tasks $\{(i, j) | 1 \leq j \leq i, i = 1, \dots, N\}$ as a *lower* triangular matrix (including the main diagonal). This entails $\frac{1}{2}N(N - 1) + N = \frac{1}{2}N(N + 1)$ operations, where $\frac{1}{2}N(N - 1)$ are off-diagonal and N are diagonal. We would like to parallelize these tasks by partitioning the atomic tasks into M subsets of rows, $\{S_m\}_{m=1, \dots, M}$, each composed of approximately $\frac{1}{2M}N(N + 1)$ tasks. The following algorithm determines the rows that constitute each subset (a molecule).

The first subset, S_1 , is composed of the first r_1 rows, that is, $S_1 = \{1, \dots, r_1\}$, for a total number of items $\frac{1}{2}r_1(r_1 + 1)$. Then, r_1 must satisfy the condition $\frac{1}{2}r_1(r_1 + 1) = \frac{1}{2M}N(N + 1)$. Solving for r_1 , we obtain the positive root

$$r_1 = \frac{-1 + \sqrt{1 + 4N(N+1)M^{-1}}}{2}$$

The second subset contains rows $S_2 = \{r_1 + 1, \dots, r_2\}$, for a total number of items $\frac{1}{2}(r_2 + r_1 + 1)(r_2 - r_1)$. Then, r_2 must satisfy the condition $\frac{1}{2}(r_2 + r_1 + 1)(r_2 - r_1) = \frac{1}{2M}N(N+1)$. Solving for r_2 , we obtain the positive root

$$r_2 = \frac{-1 + \sqrt{1 + 4(r_1^2 + r_1 + N(N+1)M^{-1})}}{2}$$

We can repeat the same argument for a future subset $S_m = \{r_{m-1} + 1, \dots, r_m\}$, with a total number of items $\frac{1}{2}(r_m + r_{m-1} + 1)(r_m - r_{m-1})$. Then, r_m must satisfy the condition $\frac{1}{2}(r_m + r_{m-1} + 1)(r_m - r_{m-1}) = \frac{1}{2M}N(N+1)$. Solving for r_m , we obtain the positive root

$$r_m = \frac{-1 + \sqrt{1 + 4(r_{m-1}^2 + r_{m-1} + N(N+1)M^{-1})}}{2}$$

And it is easy to see that r_m reduces to r_1 where $r_{m-1} = r_0 = 0$. Because row numbers are positive integers, the above results are rounded to the nearest natural number. This may mean that some partitions' sizes may deviate slightly from the $\frac{1}{2M}N(N+1)$ target. Snippet 20.6 implements this logic.

SNIPPET 20.6 THE `nestedParts` FUNCTION

```
def nestedParts(numAtoms, numThreads, upperTriang=False):
    # partition of atoms with an inner loop
    parts, numThreads_ = [0], min(numThreads, numAtoms)
    for num in xrange(numThreads_):
        part = 1 + 4 * (parts[-1]**2 + parts[-1] + numAtoms * (numAtoms + 1.) / numThreads_)
        part = (-1 + part**.5) / 2.
        parts.append(part)
    parts = np.round(parts).astype(int)
    if upperTriang: # the first rows are the heaviest
        parts = np.cumsum(np.diff(parts)[:, :-1])
        parts = np.append(np.array([0]), parts)
    return parts
```

If the outer loop iterates $i = 1, \dots, N$ and the inner loop iterates $j = i, \dots, N$, we

can order these atomic tasks $\{(i, j) | 1 \leq i \leq j, j = 1, \dots, N\}$ as an *upper* triangular matrix (including the main diagonal). In this case, the argument `upperTriang = True` must be passed to function `nestedParts`. For the curious reader, this is a special case of the bin packing problem. [Figure 20.2](#) plots a two-nested loops partition of atoms of increasing complexity into molecules. Each of the resulting 6 molecules involves a similar amount of work, even though some atomic tasks are up to 20 times harder than others.

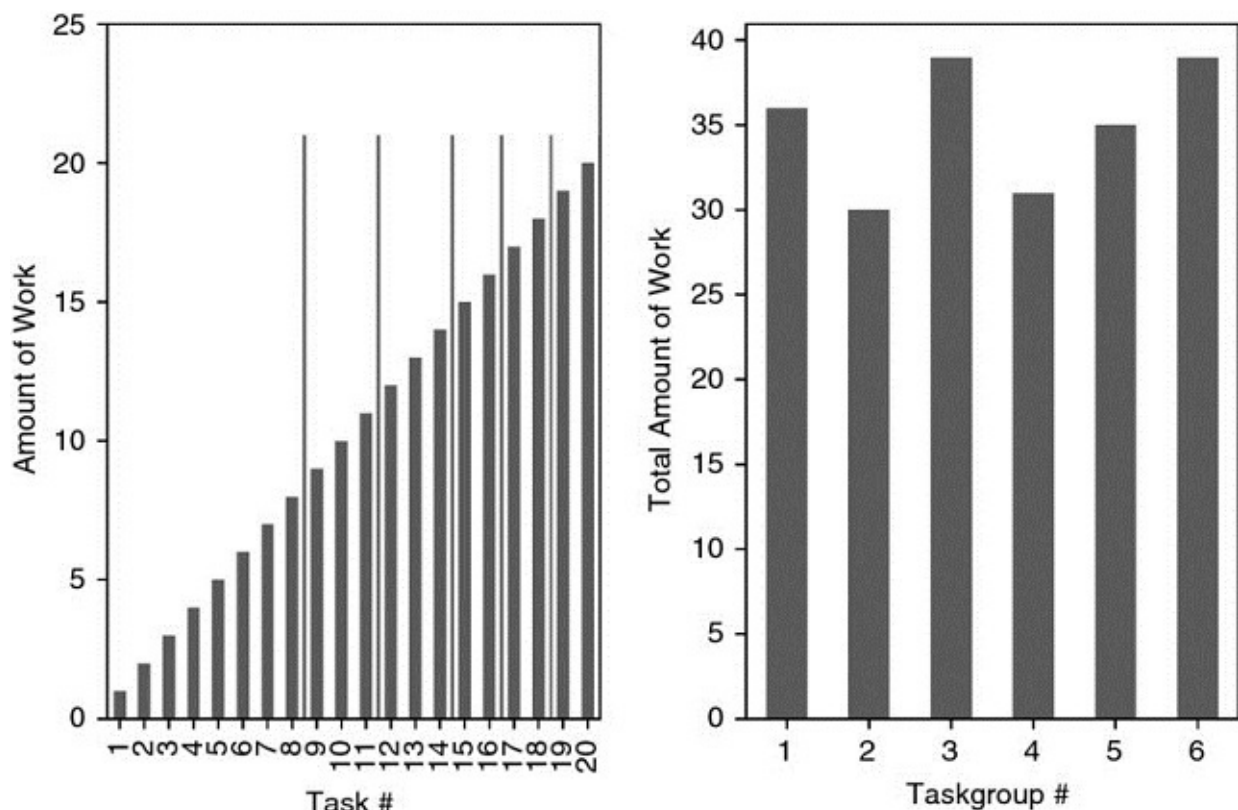


FIGURE 20.2 A two-nested loops partition of atoms into molecules

20.5 MULTIPROCESSING ENGINES

It would be a mistake to write a parallelization wrapper for each multiprocessed function. Instead, we should develop a library that can parallelize unknown functions, regardless of their arguments and output structure. That is the goal of a multiprocessing engine. In this section, we will study one such engine, and once you understand the logic, you will be ready to develop your own, including all sorts of customized properties.

20.5.1 Preparing the Jobs

In previous chapters we have made frequent use of the `mpPandasObj`. That function receives six arguments, of which four are optional:

- `func`: A callback function, which will be executed in parallel
- `pdObj`: A tuple containing:
 - The name of the argument used to pass molecules to the callback function
 - A list of indivisible tasks (atoms), which will be grouped into molecules
- `numThreads`: The number of threads that will be used in parallel (one processor per thread)
- `mpBatches`: Number of parallel batches (jobs per core)
- `linMols`: Whether partitions will be linear or double-nested
- `kargs`: Keyword arguments needed by `func`

Snippet 20.7 lists how `mpPandasObj` works. First, atoms are grouped into molecules, using `linParts` (equal number of atoms per molecule) or `nestedParts` (atoms distributed in a lower-triangular structure). When `mpBatches` is greater than 1, there will be more molecules than cores. Suppose that we divide a task into 10 molecules, where molecule 1 takes twice as long as the rest. If we run this process in 10 cores, 9 of the cores will be idle half of the runtime, waiting for the first core to process molecule 1. Alternatively, we could set `mpBatches = 10` so as to divide that task in 100 molecules. In doing so, every core will receive equal workload, even though the first 10 molecules take as much time as the next 20 molecules. In this example, the run with `mpBatches = 10` will take half of the time consumed by `mpBatches = 1`.

Second, we form a list of jobs. A job is a dictionary containing all the information needed to process a molecule, that is, the callback function, its keyword arguments, and the subset of atoms that form the molecule. Third, we will process the jobs sequentially if `numThreads = 1` (see Snippet 20.8), and in parallel otherwise (see Section 20.5.2). The reason that we want the option to run jobs sequentially is for debugging purposes. It is not easy to catch a bug when programs are run in multiple processors.¹ Once the code is debugged, we will want to use `numThreads > 1`. Fourth, we stitch together the output from every molecule into a single list, series, or dataframe.

SNIPPET 20.7 THE `mpPandasObj`, USED AT VARIOUS POINTS IN

THE BOOK

```
def
mpPandasObj(func, pdObj, numThreads=24, mpBatches=1, linMols=True, **ka
    '''
    Parallelize jobs, return a DataFrame or Series
    + func: function to be parallelized. Returns a DataFrame
    + pdObj[0]: Name of argument used to pass the molecule
    + pdObj[1]: List of atoms that will be grouped into
molecules
    + kargs: any other argument needed by func
    Example: df1=mpPandasObj(func, ('molecule
', df0.index), 24, **kargs)
    '''
    import pandas as pd
    if
linMols: parts=linParts(len(pdObj[1]), numThreads*mpBatches)
    else: parts=nestedParts(len(pdObj[1]), numThreads*mpBatches)
    jobs=[]
    for i in xrange(1, len(parts)):
        job={pdObj[0]:pdObj[1][parts[i-1]:parts[i]], 'func':func}
        job.update(kargs)
        jobs.append(job)
    if numThreads==1: out=processJobs_(jobs)
    else: out=processJobs(jobs, numThreads=numThreads)
    if isinstance(out[0], pd.DataFrame): df0=pd.DataFrame()
    elif isinstance(out[0], pd.Series): df0=pd.Series()
    else: return out
    for i in out: df0=df0.append(i)
    df0=df0.sort_index()
    return df0
```

In Section 20.5.2 we will see the multiprocessing counterpart to function `processJobs_` of Snippet 20.8.

SNIPPET 20.8 SINGLE-THREAD EXECUTION, FOR DEBUGGING

```

def processJobs_(jobs):
    # Run jobs sequentially, for debugging
    out=[]
    for job in jobs:
        out_=expandCall(job)
        out.append(out_)
    return out

```

20.5.2 Asynchronous Calls

Python has a parallelization library called multiprocessing. This library is the basis for multiprocessing engines such as joblib,² which is the engine used by many sklearn algorithms.³ Snippet 20.9 illustrates how to do an asynchronous call to Python's multiprocessing library. The reportProgress function keeps us informed about the percentage of jobs completed.

SNIPPET 20.9 EXAMPLE OF ASYNCHRONOUS CALL TO PYTHON'S MULTIPROCESSING LIBRARY

```

import multiprocessing as mp
#-----
def reportProgress(jobNum,numJobs,time0,task):
    # Report progress as asynch jobs are completed
    msg=[float(jobNum)/numJobs,(time.time()-time0)/60.]
    msg.append(msg[1]*(1/msg[0]-1))
    timeStamp=str(dt.datetime.fromtimestamp(time.time()))
    msg=timeStamp+' '+str(round(msg[0]*100,2))+'% '+task+' done after '+' \
        str(round(msg[1],2))+ ' minutes. Remaining '+str(round(msg[2],2))+ ' minutes.'
    if jobNum<numJobs:sys.stderr.write(msg+'\r')
    else:sys.stderr.write(msg+'\n')
    return
#-----
def processJobs(jobs,task=None,numThreads=24):
    # Run in parallel.
    # jobs must contain a 'func' callback, for expandCall
    if task is None:task=jobs[0]['func'].__name__
    pool=mp.Pool(processes=numThreads)
    outputs,out,time0=pool.imap_unordered(expandCall,jobs),[],time.time()
    # Process asynchronous output, report progress
    for i,out_ in enumerate(outputs,1):
        out.append(out_)
        reportProgress(i,len(jobs),time0,task)
    pool.close();pool.join() # this is needed to prevent memory leaks
    return out

```

20.5.3 Unwrapping the Callback

In Snippet 20.9, the instruction `pool.imap_unordered()` parallelized `expandCall`, by running each item in `jobs` (a molecule) in a single thread. Snippet 20.10 lists `expandCall`, which unwraps the items (atoms) in the job (molecule), and executes the callback function. This little function is the trick at the core of the multiprocessing engine: It transforms a dictionary into a task. Once you understand the role it plays, you will be able to develop your own engines.

SNIPPET 20.10 PASSING THE JOB (MOLECULE) TO THE CALLBACK FUNCTION

```
def expandCall(kargs):  
    # Expand the arguments of a callback function, kargs['func']  
    func=kargs['func']  
    del kargs['func']  
    out=func(**kargs)  
    return out
```

20.5.4 Pickle/Unpickle Objects

Multiprocessing must pickle methods in order to assign them to different processors. The problem is, bound methods are not pickleable.⁴ The work around is to add functionality to your engine, that tells the library how to deal with this kind of objects. Snippet 20.11 contains the instructions that should be listed at the top of your multiprocessing engine library. If you are curious about the precise reason this piece of code is needed, you may want to read Ascher et al. [2005], Section 7.5.

SNIPPET 20.11 PLACE THIS CODE AT THE BEGINNING OF YOUR ENGINE

```

def _pickle_method(method):
    func_name=method.im_func.__name__
    obj=method.im_self
    cls=method.im_class
    return _unpickle_method, (func_name,obj,cls)
#-----
def _unpickle_method(func_name,obj,cls):
    for cls in cls.mro():
        try:func=cls.__dict__[func_name]
        except KeyError:pass
        else:break
    return func.__get__(obj,cls)
#-----
import copy_reg,types,multiprocessing as mp
copy_reg.pickle(types.MethodType,_pickle_method,_unpickle_method)

```

20.5.5 Output Reduction

Suppose that you divide a task into 24 molecules, with the goal that the engine assigns each molecule to one available core. Function `processJobs` in Snippet 20.9 will capture the 24 outputs and store them in a list. This approach is effective in problems that do not involve large outputs. If the outputs must be combined into a single output, first we will wait until the last molecule is completed, and then we will process the items in the list. The latency added by this post-processing should not be significant, as long as the outputs are small in size and number.

However, when the outputs consume a lot of RAM, and they need to be combined into a single output, storing all those outputs in a list may cause a memory error. It would be better to perform the output reduction operation on the fly, as the results are returned asynchronously by `func`, rather than waiting for the last molecule to be completed. We can address this concern by improving `processJobs`. In particular, we are going to pass three additional arguments that determine how the molecular outputs must be *reduced* into a single output. Snippet 20.12 lists an enhanced version of `processJobs`, which contains three new arguments:

- `redux`: This is a callback to the function that carries out the reduction. For example, `redux = pd.DataFrame.add`, if output dataframes ought to be summed up.

- `reduxArgs`: This is a dictionary that contains the keyword arguments that must be passed to `redux` (if any). For example, if `redux = pd.DataFrame.join`, then a possibility is `reduxArgs = {'how': 'outer'}`.
- `reduxInPlace`: A boolean, indicating whether the `redux` operation should happen *in-place* or not. For example, `redux = dict.update` and `redux = list.append` require `reduxInPlace = True`, since appending a list and updating a dictionary are both in-place operations.

SNIPPET 20.12 ENHANCING `processJobs` TO PERFORM ON-THE-FLY OUTPUT REDUCTION

```
def processJobsRedux(jobs, task=None, numThreads=24, redux=None, reduxArgs={},
                    reduxInPlace=False):
    """
    Run in parallel
    jobs must contain a 'func' callback, for expandCall
    redux prevents wasting memory by reducing output on the fly
    """
    if task is None: task = jobs[0]['func'].__name__
    pool = mp.Pool(processes=numThreads)
    imap, out, time0 = pool.imap_unordered(expandCall, jobs), None, time.time()
    # Process asynchronous output, report progress
    for i, out_ in enumerate(imap, 1):
        if out is None:
            if redux is None: out, redux, reduxInPlace = [out_], list.append, True
            else: out = copy.deepcopy(out_)
        else:
            if reduxInPlace: redux(out, out_, **reduxArgs)
            else: out = redux(out, out_, **reduxArgs)
        reportProgress(i, len(jobs), time0, task)
    pool.close(); pool.join() # this is needed to prevent memory leaks
    if isinstance(out, (pd.Series, pd.DataFrame)): out = out.sort_index()
    return out
```

Now that `processJobsRedux` knows what to do with the outputs, we can also enhance `mpPandasObj` from Snippet 20.7. In Snippet 20.13, the new function `mpJobList` passes the three output reduction arguments to `processJobsRedux`. This eliminates the need to process an outputted list, as `mpPandasObj` did, hence saving memory and time.

SNIPPET 20.13 ENHANCING `mpPandasObj` TO PERFORM ON-THE-

FLY OUTPUT REDUCTION

```
def mpJobList (func, argList, numThreads=24, mpBatches=1, linMols=True, redux=None,
               reduxArgs={}, reduxInPlace=False, **kargs):
    if linMols: parts = linParts (len (argList [1]), numThreads*mpBatches)
    else: parts = nestedParts (len (argList [1]), numThreads*mpBatches)
    jobs = []
    for i in xrange (1, len (parts)):
        job = {argList [0]: argList [1] [parts [i-1]: parts [i]], 'func': func}
        job.update (kargs)
        jobs.append (job)
    out = processJobsRedux (jobs, redux=redux, reduxArgs=reduxArgs,
                           reduxInPlace=reduxInPlace, numThreads=numThreads)
    return out
```

20.6 MULTIPROCESSING EXAMPLE

What we have presented so far in this chapter can be used to speed-up, by several orders of magnitude, many lengthy and large-scale mathematical operations. In this section we will illustrate an additional motivation for multiprocessing: memory management.

Suppose that you have conducted a spectral decomposition of a covariance matrix of the form $Z'Z$, as we did in Chapter 8, Section 8.4.2, where Z has size $T \times N$. This has resulted in an eigenvectors matrix W and an eigenvalues matrix Λ , such that $Z'ZW = W\Lambda$. Now you would like to derive the orthogonal principal components that explain a user-defined portion of the total variance, $0 \leq \tau \leq 1$. In order to do that, we compute $P = Z\tilde{W}$, where $\tilde{W}$ contains the first $M \leq N$ columns of W , such that $(\sum_{m=1}^M \Lambda_{m,m})(\sum_{n=1}^N \Lambda_{n,n})^{-1} \geq \tau$. The computation of $P = Z\tilde{W}$ can be parallelized by noting that

$$P = Z\tilde{W} = \sum_{b=1}^B Z_b \tilde{W}_b$$

where Z_b is a sparse $T \times N$ matrix with only $T \times N_b$ items (the rest are empty), $\tilde{W}_b$ is a $N \times M$ matrix with only $N_b \times M$ items (the rest are empty), and $\sum_{b=1}^B N_b = N$. This sparsity is created by dividing the set of columns into a partition of B subsets of columns, and loading into Z_b only the b th subset of the columns. This notion of sparsity may sound a bit complicated at first, however Snippet 20.14 demonstrates how pandas allows us to implement it in a seamless way. Function

getPCs receives $\tilde{W}$ through the argument eVec. The argument molecules contains a subset of the file names in fileNames, where each file represents Z_b . The key concept to grasp is that we compute the dot product of a Z_b with the slice of the rows of $\tilde{W}_b$ defined by the columns in Z_b , and that molecular results are aggregated on the fly (redux = pd.DataFrame.add).

SNIPPET 20.14 PRINCIPAL COMPONENTS FOR A SUBSET OF THE COLUMNS

```
pcs=mpJobList(getPCs, ('molecules',fileNames), numThreads=24, mpBatches=1,
               path=path, eVec=eVec, redux=pd.DataFrame.add)
#-----
def getPCs(path,molecules,eVec):
    # get principal components by loading one file at a time
    pcs=None
    for i in molecules:
        df0=pd.read_csv(path+i,index_col=0,parse_dates=True)
        if pcs is None:pcs=np.dot(df0.values,eVec.loc[df0.columns].values)
        else:pcs+=np.dot(df0.values,eVec.loc[df0.columns].values)
    pcs=pd.DataFrame(pcs,index=df0.index,columns=eVec.columns)
    return pcs
```

This approach presents two advantages: First, because getPCs loads dataframes Z_b sequentially, for a sufficiently large B , the RAM is not exhausted. Second, mpJobList executes the molecules in parallel, hence speeding up the calculations.

In real life ML applications, we often encounter datasets where Z contains billions of datapoints. As this example demonstrates, parallelization is not only beneficial in terms of reducing run time. Many problems could not be solved without parallelization, as a matter of memory limitations, even if we were willing to wait longer.

EXERCISES

20.1 Run Snippets 20.1 and 20.2 with timeit. Repeat 10 batches of 100 executions. What is the minimum elapsed time for each snippet?

20.2 The instructions in Snippet 20.2 are very useful for unit testing, brute force searches, and scenario analysis. Can you remember where else in the book have you seen them? Where else could they have been used?

20.3 Adjust Snippet 20.4 to form molecules using a two-nested loops scheme, rather than a linear scheme.

20.4 Compare with `timeit`:

- a. Snippet 20.4, by repeating 10 batches of 100 executions. What is the minimum elapsed time for each snippet?
- b. Modify Snippet 20.4 (from exercise 3), by repeating 10 batches of 100 executions. What is the minimum elapsed time for each snippet?

20.5 Simplify Snippet 20.4 by using `mpPandasObj`.

20.6 Modify `mpPandasObj` to handle the possibility of forming molecules using a two-nested loops scheme with an upper triangular structure.

REFERENCE

Ascher, D., A. Ravenscroft, and A. Martelli (2005): *Python Cookbook*, 2nd ed. O'Reilly Media.

BIBLIOGRAPHY

Gorelick, M. and I. Ozsvald (2008): *High Performance Python*, 1st ed. O'Reilly Media.

López de Prado, M. (2017): "Supercomputing for finance: A gentle introduction." Lecture materials, Cornell University. Available at <https://ssrn.com/abstract=2907803>.

McKinney, W. (2012): *Python for Data Analysis*, 1st ed. O'Reilly Media.

Palach, J. (2008): *Parallel Programming with Python*, 1st ed. Packt Publishing.

Summerfield, M. (2013): *Python in Practice: Create Better Programs Using Concurrency, Libraries, and Patterns*, 1st ed. Addison-Wesley.

Zaccone, G. (2015): *Python Parallel Programming Cookbook*, 1st ed. Packt Publishing.

NOTES

- ¹ *Heisenbugs*, named after Heisenberg's uncertainty principle, describe bugs that change their behavior when scrutinized. Multiprocessing bugs are a prime example.
- ² <https://pypi.python.org/pypi/joblib>.
- ³ <http://scikit-learn.org/stable/developers/performance.html#multi-core-parallelism-using-joblib-parallel>.
- ⁴ <http://stackoverflow.com/questions/1816958/cant-pickle-type-instancemethod-when-using-pythons-multiprocessing-pool-ma>.